

E-Com Backend Detailed Summary

Project scanned: C:/E-Com/backend/eCom. This document explains the Spring Boot backend, its major modules, API behavior, database model, security flow, business logic, and important implementation notes.

1. High-Level Overview

- The backend is a Spring Boot e-commerce API named eCom, written in Java 21.
- It exposes REST endpoints for authentication, users, products, sellers, admin review, cart, orders, wishlist, comments, OTP signup verification, and password reset.
- Persistence is handled through Spring Data JPA with PostgreSQL. Hibernate is configured with ddl-auto=update, so entity changes can update the schema automatically during development.
- Security is JWT-based and stateless. Users log in through /auth/login, receive a token, and send it back in the Authorization: Bearer header.
- Images are not stored as binary data in the database. The backend stores image URLs and either redirects to them or returns the URL depending on the endpoint.

2. Technology Stack

Layer	Details
Runtime	Java 21, Spring Boot 4.0.0
Web/API	Spring Web MVC REST controllers
Database	PostgreSQL via Spring Data JPA and Hibernate
Security	Spring Security, BCrypt, JWT using io.jsonwebtoken
Email	Spring Mail with Gmail SMTP for OTP and reset links
Boilerplate	Lombok annotations such as @Data, @Builder, @NoArgsConstructor
Scheduling	Spring Scheduling for pending seller cleanup

3. Package Structure

- controller: REST entry points. Controllers receive HTTP requests, extract request data, and call services.
- service: Business logic. This layer handles user registration, products, orders, cart updates, wishlist changes, seller verification, OTPs, and reset links.
- model: JPA entities mapped to database tables.
- model.dto: Request and response records used to avoid exposing full entity objects.
- repo: Spring Data JPA repositories and custom query methods.
- config: Spring Security, CORS, JWT filter, and scheduling configuration.
- security: UserPrincipal adapts the User entity into Spring Security's UserDetails interface.
- scheduler: Background job that deletes expired pending sellers.
- exception: Global exception handler for duplicate email/data integrity failures.

4. API Endpoints

Area	Method / Path	What it does
Auth	GET /auth/me	Returns the authenticated user's profile from the JWT principal.
Auth	POST /auth/register	Creates a user with encoded password and selected role.
Auth	POST /auth/login	Authenticates email/password and returns JWT plus role.
Users	GET /api/me	Returns the current user profile.
Users	PUT /api/me	Updates current user's name, email, phone, address, and pin code.
Users	GET /api/user/{id}/image	Redirects to a stored user image URL.
Users	PUT /api/user/{id}/image	Stores/replaces a user image URL.
Products	GET /api/product	Returns all product summaries.
Products	GET /api/product/{id}	Returns one product summary by id.
Products	GET /api/product/{id}/image	Redirects to product image URL.
Products	GET /api/product/search	Searches name, description, or brand; optional category filter.
Products	GET /api/product/category/{category}	Returns products in the selected category.
Products	GET /api/product/{id}/rating	Returns average rating and count for a product.
Seller	GET /seller/{sellerId}/products	Returns products added by a seller.
Seller	POST /seller/products	Creates a product for the authenticated seller.
Seller	PUT /seller/product/{id}	Updates a seller product.
Seller	DELETE /seller/product/{id}	Deletes a product.
Seller	POST/PUT /seller/{id}/details	Creates or updates seller verification details.
Seller	GET /seller/{id}/details	Returns tax id, description, and verification status.
Admin	PUT /admin/sellers/{id}/approve	Approves a pending seller.
Admin	GET /admin/users	Returns all users.
Admin	GET /admin/user/{id}	Returns one user.
Admin	GET /admin/seller/requests	Lists pending seller verification requests.
Admin	GET /admin/seller/{id}/certificate	Returns seller certificate image URL.
Cart	GET /api/me/cart	Returns the authenticated customer's cart.
Cart	PUT /api/me/cart	Adds quantity to an existing cart item or creates it.
Cart	POST /api/me/cart/order	Places an order from all cart items and clears the cart.
Cart	DELETE /api/me/cart	Deletes the customer's cart.
Cart	DELETE /api/me/cart/items/{productId}	Removes one product from the cart.
Orders	GET /api/orders	Returns all orders.

Orders	POST /api/me/orders	Places an order from request items.
Orders	GET /api/orders/{id}	Returns one order.
Orders	DELETE /api/orders/{id}	Deletes one order.
Wishlist	GET /api/me/product/{id}/wishlist	Returns whether a product is in current user's wishlist.
Wishlist	GET /api/me/wishlist	Returns current user's wishlist products.
Wishlist	PUT /api/me/product/{id}/wishlist	Adds product to wishlist if not already present.
Wishlist	DELETE /api/me/product/{id}/wishlist	Removes product from wishlist.
Comments	GET /api/product/{id}/comments	Returns comments/reviews for a product.
Comments	POST /api/product/{id}/comments	Adds a review with rating and optional image URL.
Comments	PUT /api/comment/{id}	Updates an existing review.
Comments	DELETE /api/comment/{id}	Deletes a review.
OTP	POST /api/request-otp	Generates a 6-digit OTP for signup if email is not already used.
OTP	POST /api/verify-otp	Validates OTP expiry and value, then deletes it.
Password	POST /api/link-req	Creates reset token and emails reset link.
Password	POST /api/reset-password	Finds reset token, updates encoded password, deletes token.

5. Domain Model and Relationships

- User: central account entity with roles ADMIN, CUSTOMER, and SELLER. Stores name, email, phone, address, pin code, created date, encoded password, and optional image URL.
- Product: item sold on the platform. Stores name, description, price, brand, category, stock, release date, status, seller, comments, and image URL.
- SellerDetails: one-to-one profile for seller verification. Stores verification status, tax ID, business description, linked user, and certificate image URL.
- Cart: one-to-one with a user. Contains cart items with cascade and orphan removal.
- CartItem: links cart to product with quantity.
- Order: belongs to a user and contains order items. Status enum supports PENDING, PLACED, CONFIRMED, SHIPPED, DELIVERED, CANCELLED, and RETURNED.
- OrderItem: links an order to a product, storing quantity and total price at order time.
- Wishlist: one-to-one with a user and many-to-many with products through wishlist_products.
- Comment: links user and product, storing review text, rating, date, and optional review image URL.
- Otp: stores email, OTP code, expiry time, and verification flag for signup verification.
- PasswordReset: stores reset token and email for password reset flow.

6. Main Business Workflows

Registration and Login

- User posts registration data to /auth/register.
- UserService hashes the password with BCrypt and saves the chosen role.
- Login authenticates through Spring Security's AuthenticationManager.
- JwtService creates a 30-minute token containing email as subject and role as a claim.

Signup OTP Verification

- Frontend requests OTP through /api/request-otp.
- Backend rejects the request if the email already exists.
- Existing OTP for that email is removed, a new 6-digit code is generated, saved with 5-minute expiry, and emailed.
- Verification checks expiry and code, then deletes the OTP record.

Seller Verification

- Seller submits tax ID, business description, and certificate image URL.
- SellerDetails is created or updated with PENDING status by default.
- Admin lists pending sellers and approves one through /admin/sellers/{id}/approve.
- Approved sellers can add products and retrieve their own product list.

Product Listing and Search

- Public product endpoints allow fetching all products, a product by ID, image redirect, category filtering, search, and ratings.
- Seller endpoints create, update, delete, and list seller-owned products.
- Product status is computed from stock: stock greater than zero becomes AVAILABLE, otherwise OUT_OF_STOCK.

Cart and Checkout

- Cart is created lazily when a user first adds an item.
- Adding an existing product increases quantity by the requested amount.

- Ordering from cart checks stock for each product, reduces stock, creates order/order items, saves order, then clears cart.

Direct Order

- Customer posts an OrderRequest with product IDs and quantities.
- OrderService verifies stock, decreases stock, creates order items, totals price, and saves a PLACED order.

Wishlist

- Wishlist is created lazily when a product is first added.
- Adding avoids duplicates by checking if the product is already present.
- Removal searches wishlist products by product ID and throws if the product was not present.

Reviews and Ratings

- Users can add/update/delete comments with rating and optional image URL.
- Product rating endpoint calculates AVG(rating) and COUNT(comment) through repository queries.
- User-specific product review lookup supports editing a user's existing review in the frontend.

Password Reset

- User requests a reset link with email.
- Existing reset token for the email is deleted, a new UUID token is saved, and a reset link is emailed.
- Reset endpoint finds token, loads user by email, updates BCrypt password, saves user, and deletes reset record.

7. Security and Authorization

- SecurityConfig disables CSRF and uses stateless sessions, which matches JWT-based APIs.
- Public routes include /auth/**, product APIs, user image GET, OTP request/verify, reset-link request, and reset-password.
- All other routes require authentication by default.
- Role protection is applied using @PreAuthorize. /seller routes require SELLER. /admin routes require ADMIN. Some customer routes require CUSTOMER.
- UserPrincipal maps application roles to Spring authorities by prefixing ROLE_.
- JwtFilter reads Authorization: Bearer token, extracts the email subject, loads UserDetails, validates token expiry and subject, then sets SecurityContext.
- JWT expiration is 30 minutes.
- CORS allows http://localhost:5173 and https://soum1zz.github.io with credentials and GET/POST/PUT/DELETE/OPTIONS.

8. Repository Layer

Repository	Important methods
UserRepo	findById, findByEmail, existsByEmail, deleteExpiredSellers custom delete query.
ProductRepo	Search by name/description/brand with optional category; findByCategory; findBySeller_UserId.
OrderRepo	findByUser_UserId for customer order history.
CartRepo	findByUser_UserId, deleteByUser_UserId.
CartItemRepo	findByCartItemIdAndProductId, though current service mostly uses cart item stream search.
CommentRepo	Find by product, user, user+product; calculate average rating and total rating count.
WishlistRepo	findByUser_UserId and existence check by user/product relation.
OtpRepo	findByEmail and deleteByEmail.
PRRepo	findByToken and deleteByEmail for password reset records.
sellerDetailsRepo	findByUser_UserId and findByVerificationStatus.

9. Configuration

- application.properties configures a Neon PostgreSQL database connection, JWT secret, Hibernate ddl-auto=update, SQL logging, multipart support, web DEBUG logging, and Gmail SMTP.
- The project stores database and mail credentials directly in application.properties. That works locally, but production should move secrets to environment variables or a secrets manager.
- Spring Mail is used for both OTP and password reset messages.
- Scheduling is enabled globally by SchedulingConfig.

10. Scheduled Job

- SellerCleanupJob runs every day at 2:00 AM.
- It calls UserRepo.deleteExpiredSellers(PENDING, LocalDateTime.now().minusDays(7)).
- Intent: remove seller users who remain pending for more than 7 days.

- Implementation note: User.createdOn is LocalDate, while the repository query compares it with LocalDateTime. This may need checking because date/time type mismatch can cause runtime query issues.

11. Important Observations and Risks

- Secrets are committed in application.properties: database URL/username/password, JWT secret, and Gmail app password. These should be rotated and moved out of source control.
- Some endpoints use @PreAuthorize("hasRole('USER')") for comments, but the UserRole enum has ADMIN, CUSTOMER, SELLER. Unless there is a separate USER role, comment create/update may be blocked.
- Seller product update/delete endpoints accept product ID but do not verify that the authenticated seller owns that product. A seller could potentially update another seller's product if they know the ID.
- Admin user image endpoint is /admin/user/{id}/image while public/user image endpoint is /api/user/{id}/image; the behavior differs: one returns URL, one redirects.
- JwtFilter has a path check for /api/user/*/image using equals, which will not match wildcard paths. SecurityConfig already permits the route, so this check may be unnecessary.
- ProductResponse does not include image URL or release date. Frontend must call image redirect endpoint and cannot prefill release date in update forms unless another endpoint is added.
- Several controllers catch exceptions and return generic BAD_REQUEST, which can make frontend debugging harder.
- Cart update adds requested quantity to existing quantity. If the frontend sends the desired final quantity, backend will over-increment; currently it behaves like an add-to-cart action.
- Order stock is reduced but ProductService.updateProduct handles status only during product updates; order placement reduces stock but does not set OUT_OF_STOCK when stock becomes zero.
- CrossOrigin annotations are inconsistent: some use http://localhost:5173, some https://localhost:5173, while global CORS allows http://localhost:5173 and GitHub Pages.

12. End-to-End Mental Model

Mercato's backend is centered around a User account and role-driven access. Customers browse public products, authenticate, manage profile/cart/wishlist, place orders, and review products. Sellers register as SELLER, submit verification details, wait for admin approval, then create and manage products. Admins review pending seller details and approve them. JWT authentication connects every protected request back to the current User entity. JPA repositories provide the persistence layer, while service classes hold most of the business behavior.

Generated from local source scan of C:/E-Com/backend/eCom.